



**Projektová dokumentácia**  
**Implementácia prekladača imperatívneho jazyka IFJ23**  
Tím xjerab28, varianta vv-BVS

6. decembra 2023

|                      |                   |      |
|----------------------|-------------------|------|
| <b>Jakub Jeřábek</b> | <b>(xjerab28)</b> | 25 % |
| Vojtěch Teichmann    | (xteich02)        | 25 % |
| Doubravka Šimůnková  | (xsimun05)        | 25 % |
| Monika Záhradníková  | (xzahra33)        | 25 % |

# Obsah

|          |                                                                         |          |
|----------|-------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Úvod</b>                                                             | <b>1</b> |
| <b>2</b> | <b>Návrh a implementácia</b>                                            | <b>1</b> |
| 2.1      | Lexikálna analýza . . . . .                                             | 1        |
| 2.2      | Syntaktická a sémantická analýza . . . . .                              | 1        |
| 2.2.1    | Spracovanie výrazov pomocou precedenčnej syntaktickej analýzy . . . . . | 2        |
| 2.3      | Generátor kódu IFJcode23 . . . . .                                      | 2        |
| <b>3</b> | <b>Použité algoritmy a datové štruktúry</b>                             | <b>3</b> |
| 3.1      | Tabuľka symbolov . . . . .                                              | 3        |
| 3.2      | Zásobník symbolov pre precedenčnú syntaktickú analýzu . . . . .         | 3        |
| 3.3      | Zásobník tabuliek symbolov . . . . .                                    | 3        |
| 3.4      | Dynamický reťazec . . . . .                                             | 3        |
| <b>4</b> | <b>Práca v tíme</b>                                                     | <b>3</b> |
| 4.1      | Spôsob práce v tíme a komunikácia . . . . .                             | 3        |
| 4.2      | Verzovací systém . . . . .                                              | 4        |
| 4.3      | Rozdelenie práce medzi členov . . . . .                                 | 4        |
| 4.4      | Problémy . . . . .                                                      | 5        |
| <b>5</b> | <b>Záver</b>                                                            | <b>5</b> |
| 5.1      | Zdroje . . . . .                                                        | 5        |
| <b>6</b> | <b>Prílohy</b>                                                          | <b>6</b> |

# 1 Úvod

Cieľom projektu bolo implementovať prekladač imperatívneho jazyka IFJ23 do predmetov IFJ a IAL. Zadaným jazykom na implementáciu bol jazyk C. Program funguje ako konzolová aplikácia, ktorá načíta zdrojový kód napísaný v jazyku IFJ23 a preloží ho do cieľového jazyka IFJcode23 (medzikód). V prípade chyby vracia program odpovedajúcu návratovú hodnotu.

## 2 Návrh a implementácia

Projekt je zostavený zo štyroch základných častí, ktoré budú vysvetlené v nasledujúcich podkapitolách.

### 2.1 Lexikálna analýza

Začali sme implementáciou lexikálneho analyzátora, ktorý sa nachádza v súbore `scanner.c`. Lexikálny analyzátor číta zo štandardného vstupu, ktorý musí obsahovať program napísaný v jazyku IFJ23. Základom lexikálnej analýzy je deterministický konečný automat (ďalej DKA), ktorého diagram je na stranách 6 a 7.

Hlavnou funkciou lexikálneho analyzátora je funkcia `getToken`, ktorá vyžaduje parametre `token` a `swiftFile`. Parameter `swiftFile` je ukazateľ na vstupný súbor obsahujúci program napísaný v jazyku IFJ23 a `token` je ukazateľ na datovú štruktúru `Token`.

Funkcia číta každý znak a cez príkaz `switch` prechádza do nasledujúcich stavov (to je definované pomocou DKA). Keď príkaz `switch` prejde do stavu, ktorý odpovedá aktuálnemu tokenu (nájde lexikálne správny token), uloží do parametra `token` typ tokenu (typy sú definované výčtovým typom `TokenType`) a hodnotu tokenu. Analyzátor rozlišuje, či je hodnota tokenu datového typu `string`, `int`, `double`, či je to kľúčové slovo alebo identifikátor. Na základe toho volá funkciu s prefixom `process`, ktorá token ďalej spracuje.

Spracovanie escape sekvencií vykonáva funkcia `processString`, ktorej základom je taktiež konečný automat.

Po dokončení činnosti hlavnej funkcie `getToken` sa volá funkcia `handleError`, ktorá vráti odpovedajúci návratový kód a uvoľní alokované prostriedky alebo, v prípade chyby, program ukončí.

### 2.2 Syntaktická a sémantická analýza

Ďalej sme pokračovali implementáciou syntaktického a sémantického analyzátora, ktoré sa nachádzajú v súbore `parser.c`. Syntaktická analýza je založená na LL – gramatike (nájdete na strane 9) a riadi sa metódou rekurzívneho zostupu. Týmto postupom sa riadi celá syntaktická analýza okrem spracovania výrazov (bližšie v podkapitole 2.2.1). Syntaktický a sémantický analyzátor pracujú zároveň.

Syntaktický analyzátor získava tokeny prostredníctvom lexikálnej analýzy volaním funkcie `getToken`. Na základe typu tokena `TokenType` sa syntaktický analyzátor rozhoduje, ktoré pravidlo LL – gramatiky použije.

Sémantický analyzátor vykonáva typové kontroly a pracuje s tabuľkou symbolov (bližšie popísaná v podkapitole 3.1). Volá operáciu `symtableInsert`, ktorá ukladá do tabuľky symbolov identifikátory spolu s ich vlastnosťami. Zároveň pracuje aj so zásobníkom tabuliek symbolov `symStack` (bližšie popísané v podkapitole 3.3). Pre každý rámec vytvorí jednu tabuľku symbolov a uloží ju na vrchol zásobníka. Pokiaľ chce nájsť daný identifikátor v zásobníku, volá operáciu `symstackSearchId`, ktorá prechádza všetky rámce postupne od vrcholu zásobníka až ku dnu zásobníka.

Hlavná funkcia celého prekladača `main` sa nachádza v súbore `parser.c`. Tá najprv volá funkciu `initParser`, ktorá inicializuje parser, tabuľku symbolov pre globálny rámec a zásobník tabuliek symbolov. Potom funkcia `initParser` uloží do tabuľky symbolov všetky vstavané funkcie jazyka IFJ23 za pomoci funkcie `buildInFunctions`. Keď funkcia `initParser` skončí, funkcia `main` zavolá funkciu `program`.

Program je najdôležitejšou funkciou v súbore `parser.c`. Vyžaduje ako parameter vstupný súbor napísaný v jazyku IFJ23. Funkcia `program` na začiatku vykoná prvý priechod vstupným súborom volaním

funkcie `firstPassage`. V prvom priechode sa prechádza vstupný súbor a globálny rámec sa naplní deklaráciami funkcií. Súčasne sa prechádza aj konštrukciami `while` cyklov, ktorých premenné deklarované vnútri cyklu sa ukladajú do dynamicky alokovaného pola `whileDefinitions`.

Po prvom priechode sa tabuľka symbolov pre globálny rámec uloží na vrchol zásobníka a inicializuje sa generátor (bližšie popísaný v podkapitole 2.3). Teraz je parser pripravený na druhý priechod vstupným súborom.

Spracúva telo vstupného súboru volaním rekurzívnej funkcie `sequenceOrFunction`. Názov funkcie vypovedá o tom, že syntaktický analyzátor rozlišuje, či aktuálne spracúva deklaráciu funkcie alebo sekvenciu príkazov. Pokiaľ sa jedná o sekvenciu príkazov, volá sa rekurzívna funkcia `sequence`, v ktorej sa syntaktický analyzátor podľa typu tokenu rozhoduje, ktorú funkciu zavolá. Zároveň s ním samozrejme pracuje aj sémantický analyzátor, ktorý je popísaný vyššie.

V prípade, že syntaktický analyzátor narazí na miesto, kde očakáva výraz, predá riadenie precedenčnému syntaktickému analyzátoru implementovanému v súbore `expression.c` (bližšie v podkapitole 2.2.1).

Ak syntaktický analyzátor dostáva token, ktorý nevie spracovať na základe pravidiel LL – gramatiky, ukončuje program vstavanou funkciou v jazyku C `exit` s príslušným návratovým kódom.

Pokiaľ sémantický analyzátor narazí na chybu, taktiež ukončuje program funkciou `exit` s príslušným návratovým kódom.

### 2.2.1 Spracovanie výrazov pomocou precedenčnej syntaktickej analýzy

Ako sme vyššie spomenuli, výrazy spracuje precedenčný syntaktický analyzátor, ktorý je napísaný v súbore `expression.c`. Súčasne s ním vykonáva typové kontroly sémantický analyzátor. Na začiatku súboru je implementovaná precedenčná tabuľka, ktorú máte možnosť vidieť na strane 8.

Výrazy sa spracujú metódou konvertovania infixu na postfix za pomoci zásobníka implementovaného v súbore `stack.c` (bližšie popísané v podkapitole 3.2). Hlavnou funkciou je funkcia `createExpression`, ktorá je volaná parserom a vracia mu datovú štruktúru `expressionStruct` so spracovaným výrazom.

Pokiaľ sémantický analyzátor narazí na nezhodnosť datových typov, ukončuje program vstavanou funkciou `exit` s príslušným návratovým kódom.

## 2.3 Generátor kódu IFJcode23

Generovanie výstupného kódu IFJcode23 je poslednou časťou nášho prekladača. Generovanie je implementované v súbore `codeGenerator.c`. Celé generovanie kódu ovláda parser a volá funkcie implementované v `codeGenerator.c`, aby sa vygeneroval výsledný kód pre určitú sekvenciu príkazov, ktorú aktuálne parser spracoval.

Ako príklad môžeme uviesť situáciu, kedy parser aktuálne spracúva definíciu funkcie. Po spracovaní hlavičky funkcie zavolá parser funkciu `generateFunctionHead` a ďalej spracúva telo funkcie. Po spracovaní tela volá parser funkciu `generateFunctionEnd`. V tomto čase by mala byť vygenerovaná celá funkcia. Obdobne parser volá generátor pri všetkých príkazoch jazyka IFJ23.

V našom projekte sme nepoužili trojadresný kód, ale prevod na postfixový tvar. Pokiaľ parser aktuálne spracúva výraz a príde čas na generovanie kódu, zavolá funkciu `generateExpression` s parametrom `expression`, čo je štruktúra obsahujúca postfixový výraz a informácie o premenných.

Pri generovaní premenných, či návěstí podmienok a cyklov je ako ďalší parameter predávaný `labelIndex`, ktorý slúži k univerzálnej identifikácii premenných, i keď je premenná deklarovaná v globálnom a zároveň v lokálnom rámci. To umožňuje efektívnu prácu so všetkými premennými, ktoré se môžu nachádzať v rôznych rámcoch.

V súbore `codeGenerator.c` sú implementované aj dve dôležité funkcie `comment` a `inst`, ktoré generátor využíva na výpis komentárov a jednotlivých inštrukcií.

## 3 Použité algoritmy a datové štruktúry

### 3.1 Tabuľka symbolov

Tabuľku symbolov sme implementovali v súbore `symtable.c` podľa varianty zadania, ktoré sme si vybrali. Zvolili sme výškovo-vyvážený binárny vyhľadávací strom (ďalej BST). Pri implementácii BST sme postupovali podľa prednášaných techník z predmetu IAL.

Vytvorili sme si datovú štruktúru `Tnode`, ktorá popisovala uzol BST. V každom uzle sme ukladali názov identifikátora ako kľúč `key`. Všetky vlastnosti identifikátora, ktoré sémantická analýza potrebovala pre prácu s identifikátorom, sme ukladali do datovej štruktúry `Tdata`.

### 3.2 Zásobník symbolov pre precedenčnú syntaktickú analýzu

Na to, aby mohla precedenčná syntaktická analýza konvertovať výraz zadaný v infixovom tvare na tvar postfixový, sme potrebovali implementovať zásobník. Tu sa symboly ukladajú na vrchol zásobníka a naopak odstraňujú z vrcholu zásobníka na základe pravidiel precedenčnej tabuľky (na strane 8), ktorá hovorí o tom, ktorý symbol má vyššiu prioritu.

Zásobník je implementovaný v súbore `stack.c` a pri implementácii sme sa nechali inšpirovať metódami prebranými na prednáškach predmetu IAL.

### 3.3 Zásobník tabuliek symbolov

Ďalší zásobník, ktorý sme museli implementovať, bol zásobník tabuliek symbolov, ktorý sa nachádza v súbore `symstack.c`. Potrebovali sme ho, aby sémantický analyzátor dokázal rozlišovať, ktoré identifikátory (premenné) sa nachádzajú v ktorom rámci. Každý prvok zásobníka je vlastne odkaz na tabuľku symbolov pre globálny alebo lokálny rámec.

So zásobníkom sa dajú vykonávať klasické operácie preberané aj na prednáškach z IAL. Výnimkou je funkcia `symstackSearchId`, ktorá bola už popísaná vyššie v podkapitole 2.2.

### 3.4 Dynamický reťazec

Dynamický reťazec a operácie nad ním sme implementovali v súbore `string.c`. Vytvorili sme štruktúru `String`, do ktorej sa postupne ukladá reťazec tokenov, o ktorom lexikálna analýza dopredu nevie, koľko bude mať znakov. Ďalej sa do štruktúry ukladá veľkosť už alokovanej pamäte pre reťazec a aktuálna dĺžka reťazca.

Z operácií nad dynamickým stringom môžeme spomenúť operáciu `addCharStr`, ktorá pridá na koniec dynamického reťazca nový znak alebo operáciu `addStrStr`, ktorá pridá na koniec dynamického reťazca nový reťazec.

Operácie nad dynamickým stringom volá lexikálna analýza a generátor kódu.

## 4 Práca v tíme

### 4.1 Spôsob práce v tíme a komunikácia

Na začiatku projektu sme sa osobne stretli a prediskutovali, akým spôsobom budeme pracovať. Stanovili sme si, že chceme mať projekt hotový už pred pokusným odovzdávaním. Dohodli sme sa na stretávaní minimálne raz týždenne, kde sme si vždy zhrnuli, ako sme pokročili v práci, poprípade sme programovali a snažili sa nájsť riešenie problémov. Pokiaľ nebolo možné osobné stretnutie, stretli sme sa online cez hovor, prostredníctvom Discordu.

Pre prehľadnosť informácií ohľadom projektu (hlavne v začiatkoch) sme využívali software Padlet, kde sme si vytvorili zdieľanú nástenku a ukladali sem všetky potrebné informácie.

## 4.2 Verzovací systém

Pre správu verzií sme využili verzovací systém Git, s ktorým sme už mali skúsenosti. GitHub nám umožnil vytvoriť zdieľaný repozitár, a tak sme mohli pracovať na viacerých úlohách súčasne. Zo začiatku sme pracovali na hlavnej vetve, neskôr sme prešli k práci na vedľajších vetvách, ktoré nám poskytovali istotu, že prípadné problémy nebudú zavedené do už fungujúceho programu.

## 4.3 Rozdelenie práce medzi členov

Rozdelenie práce neprebehlo hneď na začiatku, ale vyvíjalo sa postupne pri práci na projekte. Práca bola rozdelená nasledovne:

### **Jakub Jeřábek:**

- Vedenie tímu
- Implementácia lexikálneho analyzátora
- Implementácia parseru
- Implementácia generátora kódu
- Testovanie

### **Doubravka Šimůnková:**

- Návrh LL – gramatiky
- Návrh LL – tabuľky
- Návrh precedenčnej tabuľky
- Implementácia parseru
- Testovanie

### **Vojtěch Teichmann:**

- Návrh automatu pre lexikálnu analýzu
- Implementácia spracovania výrazov
- Testovanie

### **Monika Záhradníková:**

- Organizácia stretnutí
- Implementácia tabuľky symbolov
- Implementácia zásobníka symbolov
- Testovanie
- Dokumentácia, prezentácia

## 4.4 Problémy

Ako v každom tímovom projekte, aj u nás sa vyskytlo zopár problémov, ktoré sme nevedeli hneď vyriešiť.

Problém nám robili deklarácie premenných vo `while` cykloch, čo sme vyriešili spracovaním týchto deklarácií už v prvom priechode programom.

Ďalším problémom bolo napojenie tabuľky symbolov a zásobníka tabuliek symbolov na parser. Program nám veľa krát hlásil chybu `segmentation fault` tj. neoprávnený prístup do pamäte. Nakoniec sme to vyriešili spoločne na stretnutí.

Problém nám robil taktiež fakt, že znak `EOL` sa môže nachádzať na akomkoľvek mieste vo vstupnom súbore. Riešením bolo pridanie tokenu `EOL` do výčtového typu `TokenType` a zmena `LL` – gramatiky.

Nakoniec by sme ako problém spomenuli prípady, kedy parser nedokázal jednoznačne určiť len na základe aktuálneho tokenu, ktoré pravidlo má použiť. V niektorých prípadoch sa napríklad parser potreboval pozrieť do tabuľky symbolov, aby mohol vybrať pravidlo `LL` – gramatiky na základe toho, či je aktuálny identifikátor identifikátorom premennej alebo funkcie. Kvôli tomuto problému sme do `LL` – gramatiky zaviedli nedeterministické pravidlá, ktoré sú v `LL` – gramatike (na strane 9) vyznačené na červeno.

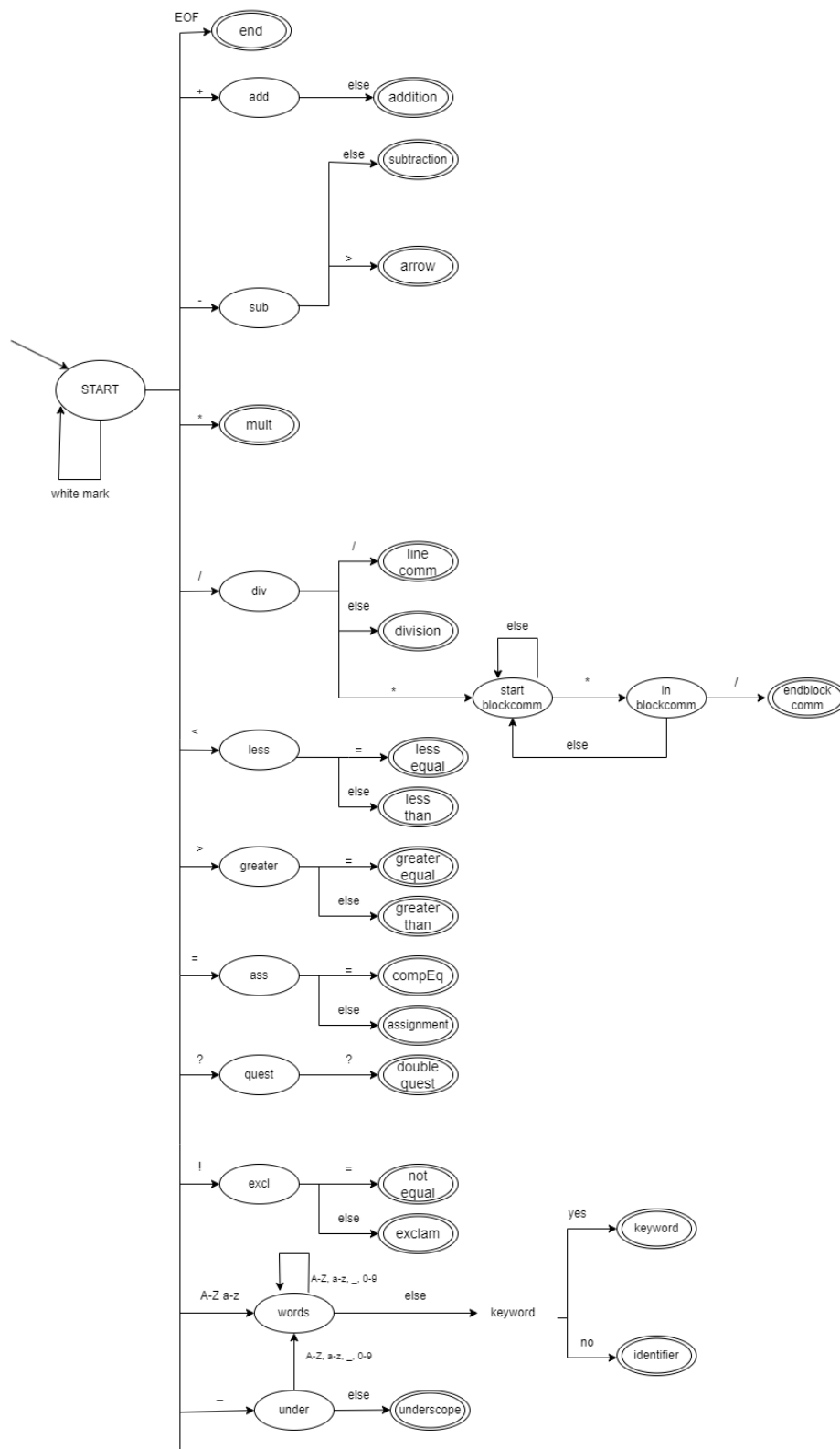
## 5 Záver

Zo začiatku bolo veľmi náročné preniesť všetky informácie z prednášaných látok do projektu a predstaviť si, ako to má celé fungovať. Postupom času sme prišli na spôsob, ako implementovať jednotlivé časti tak, aby spolu dokázali komunikovať a vytvoriť tak celok, ktorým je prekladač. Určite nás to veľa naučilo a dalo nám kopec skúseností či už v samotnom programovaní, v problematike prekladačov alebo v tímovej práci.

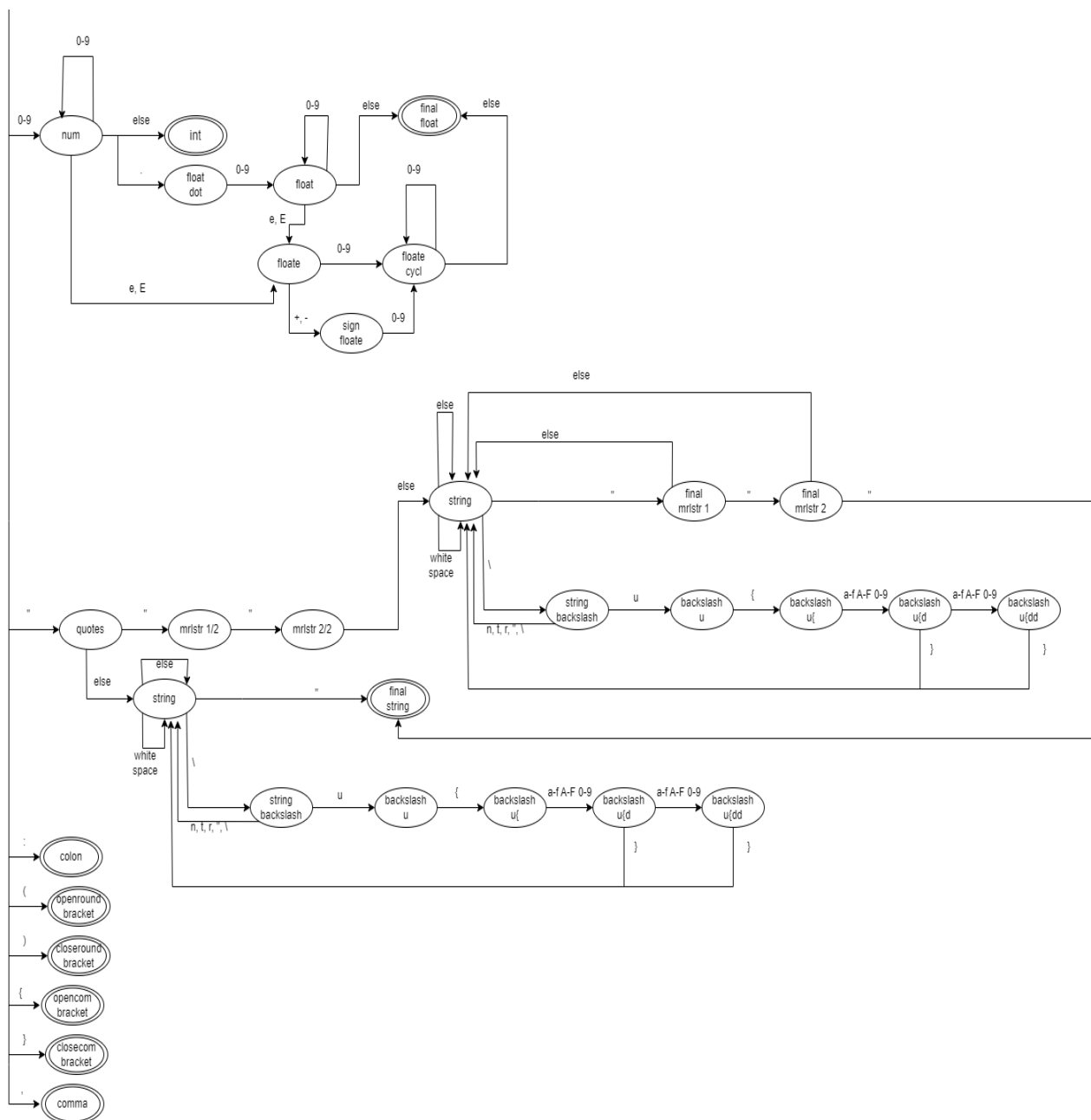
### 5.1 Zdroje

Ako zdroje sme využívali výukové materiály predmetov IFJ a IAL. Informácie sme čerpali hlavne z demo cvičení, prednášok a prezentácií.

## 6 Prílohy



Obr. 1: Diagram konečného automatu špecifikujúci lexikálnu analýzu (1. časť)



Obr. 2: Diagram konečného automatu špecifikujúci lexikálnu analýzu (2. časť)

|            | + | - | * | / | < | > | <= | >= | == | != | ! | ?? | ( | ) | id | cel<br>cis | des<br>cis | řet | nil | \$ |
|------------|---|---|---|---|---|---|----|----|----|----|---|----|---|---|----|------------|------------|-----|-----|----|
| +          | > | > | < | < | > | > | >  | >  | >  | >  | < | >  | < | > | <  | <          | <          | <   |     | >  |
| -          | > | > | < | < | > | > | >  | >  | >  | >  | < | >  | < | > | <  | <          | <          |     |     | >  |
| *          | > | > | > | > | > | > | >  | >  | >  | >  | < | >  | < | > | <  | <          | <          |     |     | >  |
| /          | > | > | > | > | > | > | >  | >  | >  | >  | < | >  | < | > | <  | <          | <          |     |     | >  |
| <          | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   |     | >  |
| >          | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   |     | >  |
| <=         | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   |     | >  |
| >=         | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   |     | >  |
| ==         | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   | <   | >  |
| !=         | < | < | < | < |   |   |    |    |    |    | < | >  | < | > | <  | <          | <          | <   | <   | >  |
| !          | > | > | > | > | > | > | >  | >  | >  | >  |   | >  |   | > |    |            |            |     |     | >  |
| ??         | < | < | < | < | < | < | <  | <  | <  | <  | < | <  | < | < | >  | <          | <          | <   | <   | >  |
| (          | < | < | < | < | < | < | <  | <  | <  | <  | < | <  | < | = | <  | <          | <          | <   | <   |    |
| )          | > | > | > | > | > | > | >  | >  | >  | >  | > | >  |   | > |    |            |            |     |     | >  |
| id         | > | > | > | > | > | > | >  | >  | >  | >  | > | >  |   | > |    |            |            |     |     | >  |
| cel<br>cis | > | > | > | > | > | > | >  | >  | >  | >  |   |    |   | > |    |            |            |     |     | >  |
| des<br>cis | > | > | > | > | > | > | >  | >  | >  | >  |   |    |   | > |    |            |            |     |     | >  |
| řet        | > |   |   |   | > | > | >  | >  | >  | >  |   |    |   | > |    |            |            |     |     | >  |
| nil        |   |   |   |   |   |   |    |    | >  | >  |   |    |   | > |    |            |            |     |     | >  |
| \$         | < | < | < | < | < | < | <  | <  | <  | <  | < | <  | < |   | <  | <          | <          | <   | <   |    |

Obr. 3: Precedenčná tabuľka špecifikujúca precedenčnú syntaktickú analýzu pre spracovanie výrazov

|     |                      |                                                                                                                                   |
|-----|----------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| 1.  | [program]            | [enter] [sequenceOrFunction] EOF                                                                                                  |
| 2.  | [sequenceOrFunction] | epsilon                                                                                                                           |
| 3.  | [sequenceOrFunction] | [sequence] [sequenceOrFunction]                                                                                                   |
| 4.  | [sequenceOrFunction] | Func [enter] id [enter] ([enter]<br>[listOfParams]) [enter] [returnType] {[enter]<br>[sequence]} EOL [enter] [sequenceOrFunction] |
| 5.  | [sequence]           | epsilon                                                                                                                           |
| 6.  | [sequence]           | [initType] [enter] id [enter] [determineType]<br>EOL [enter] [sequence]                                                           |
| 7.  | [sequence]           | If [enter] [statement] {[enter] [sequence]}<br>[enter] else [enter] {[enter] [sequence]} EOL<br>[enter] [sequence]                |
| 8.  | [sequence]           | While [enter] [expression] {[enter] [sequence]}<br>EOL [enter] [sequence]                                                         |
| 9.  | [sequence]           | Id [enter] = [enter] [valueToVariable] EOL<br>[enter] [sequence]                                                                  |
| 10. | [sequence]           | [callingFunction] EOL [enter] [sequence]                                                                                          |
| 11. | [sequence]           | Return [returnExpression] EOL [enter]<br>[sequence]                                                                               |
| 12. | [initType]           | Var                                                                                                                               |
| 13. | [initType]           | Let                                                                                                                               |
| 14. | [returnExpression]   | epsilon                                                                                                                           |
| 15. | [returnExpression]   | [expression]                                                                                                                      |
| 16. | [listOfParams]       | epsilon                                                                                                                           |
| 17. | [listOfParams]       | [parameter] [parameters]                                                                                                          |
| 18. | [parameters]         | , [parameter] [parameters]                                                                                                        |
| 19. | [parameters]         | epsilon                                                                                                                           |
| 20. | [parameter]          | [name] [name] : [type] [enter]                                                                                                    |
| 21. | [name]               | Id [enter]                                                                                                                        |
| 22. | [name]               | _ [enter]                                                                                                                         |
| 23. | [returnType]         | epsilon                                                                                                                           |
| 24. | [returnType]         | -> [enter] [type]                                                                                                                 |
| 25. | [type]               | Double                                                                                                                            |
| 26. | [type]               | Double?                                                                                                                           |
| 27. | [type]               | Int                                                                                                                               |
| 28. | [type]               | Int?                                                                                                                              |
| 29. | [type]               | String                                                                                                                            |
| 30. | [type]               | String?                                                                                                                           |
| 31. | [determineType]      | = [enter] [valueToVariable]                                                                                                       |
| 32. | [determineType]      | : [enter] [type] [possibleExpression]                                                                                             |
| 33. | [possibleExpression] | = [enter] [valueToVariable]                                                                                                       |
| 34. | [possibleExpression] | epsilon                                                                                                                           |
| 35. | [statement]          | [expression]                                                                                                                      |
| 36. | [statement]          | Let [enter] id [enter]                                                                                                            |
| 37. | [valueToVariable]    | [expression]                                                                                                                      |
| 38. | [valueToVariable]    | [callingFunction]                                                                                                                 |

Tabuľka 1: LL – gramatika riadiaca syntaktickú analýzu (1. časť)

|     |                     |                                              |
|-----|---------------------|----------------------------------------------|
| 39. | [callingFunction]   | Id [enter] ([enter] [listOfEnterParams])     |
| 40. | [listOfEnterParams] | epsilon                                      |
| 41. | [listOfEnterParams] | [enterParameter] [enterParameters]           |
| 42. | [enterParameters]   | , [enter] [enterParameter] [enterParameters] |
| 43. | [enterParameters]   | epsilon                                      |
| 44. | [enterParameter]    | [nameParameter] [term]                       |
| 45. | [nameParameter]     | Id [enter] : [enter]                         |
| 46. | [nameParameter]     | epsilon                                      |
| 47. | [term]              | id                                           |
| 48. | [term]              | Deset.cislo                                  |
| 49. | [term]              | Cele.cislo                                   |
| 50. | [term]              | retezec                                      |
| 51. | [term]              | nil                                          |
| 52. | [enter]             | epsilon                                      |
| 53. | [enter]             | EOL[enter]                                   |

Tabuľka 2: LL – gramatika riadiaca syntaktickú analýzu (2. časť)

|                      | E<br>O<br>F | E<br>O<br>L | id            | fu<br>nc | va<br>r | le<br>t | w<br>hi<br>le | if | {  | }  | (  | )  | D<br>e<br>s<br>c<br>e<br>n<br>d<br>e<br>r | C<br>e<br>l<br>e<br>s<br>t<br>e<br>r | r<br>e<br>t<br>u<br>r<br>n | -  | ,  | -> | D<br>o<br>u<br>b<br>l<br>e | D<br>o<br>u<br>b<br>l<br>e? | In<br>t | In<br>t? | St<br>r<br>i<br>n<br>g | St<br>r<br>i<br>n<br>g? | =  | :  |    |
|----------------------|-------------|-------------|---------------|----------|---------|---------|---------------|----|----|----|----|----|-------------------------------------------|--------------------------------------|----------------------------|----|----|----|----------------------------|-----------------------------|---------|----------|------------------------|-------------------------|----|----|----|
| [program]            | 1           | 1           | 1             | 1        | 1       | 1       | 1             | 1  |    |    |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [sequenceOrFunction] | 2           |             | 3             | 4        | 3       | 3       | 3             | 3  |    |    |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [sequence]           | 5           |             | 9/<br>10      | 5        | 6       | 6       | 8             | 7  |    | 5  |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [initType]           |             |             |               |          | 12      | 13      |               |    |    |    |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [returnExpression]   |             | 14          | 15            |          |         |         |               |    |    | 14 | 15 |    | 15                                        | 15                                   | 15                         | 15 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [listOfParams]       |             |             | 17            |          |         |         |               |    |    |    |    | 16 |                                           |                                      |                            |    | 17 |    |                            |                             |         |          |                        |                         |    |    |    |
| [parameters]         |             |             |               |          |         |         |               |    |    |    |    | 19 |                                           |                                      |                            |    |    | 18 |                            |                             |         |          |                        |                         |    |    |    |
| [parameter]          |             |             | 20            |          |         |         |               |    |    |    |    |    |                                           |                                      |                            |    | 20 |    |                            |                             |         |          |                        |                         |    |    |    |
| [name]               |             |             | 21            |          |         |         |               |    |    |    |    |    |                                           |                                      |                            |    | 22 |    |                            |                             |         |          |                        |                         |    |    |    |
| [returnType]         |             |             |               |          |         |         |               |    | 23 |    |    |    |                                           |                                      |                            |    |    |    | 24                         |                             |         |          |                        |                         |    |    |    |
| [type]               |             |             |               |          |         |         |               |    |    |    |    |    |                                           |                                      |                            |    |    |    |                            | 25                          | 26      | 27       | 28                     | 29                      | 30 |    |    |
| [determineType]      |             |             |               |          |         |         |               |    |    |    |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    | 31 | 32 |
| [possibleExpression] | 34          | 34          |               |          |         |         |               |    |    | 34 |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    | 33 |    |
| [statement]          |             |             | 35            |          | 36      |         |               |    |    |    | 35 |    | 35                                        | 35                                   | 35                         | 35 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [valueToVariable]    |             |             | 37<br>/3<br>8 |          |         |         |               |    |    |    | 37 |    | 37                                        | 37                                   | 37                         | 37 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [callingFunction]    |             |             | 39            |          |         |         |               |    |    |    |    |    |                                           |                                      |                            |    |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [listOfEnterParams]  |             |             | 41            |          |         |         |               |    |    |    |    | 40 | 41                                        | 41                                   | 41                         | 41 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [enterParameters]    |             |             |               |          |         |         |               |    |    |    |    | 43 |                                           |                                      |                            |    |    | 42 |                            |                             |         |          |                        |                         |    |    |    |
| [enterParameter]     |             |             | 44            |          |         |         |               |    |    |    |    |    | 44                                        | 44                                   | 44                         | 44 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [nameParameter]      |             |             | 45<br>/4<br>6 |          |         |         |               |    |    |    |    |    | 46                                        | 46                                   | 46                         | 46 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [term]               |             |             | 47            |          |         |         |               |    |    |    |    |    | 48                                        | 49                                   | 50                         | 51 |    |    |                            |                             |         |          |                        |                         |    |    |    |
| [enter]              | 52          | 53          | 52            | 52       | 52      | 52      | 52            | 52 | 52 | 52 | 52 | 52 | 52                                        | 52                                   | 52                         | 52 | 52 | 52 | 52                         | 52                          | 52      | 52       | 52                     | 52                      | 52 | 52 | 52 |

Obr. 4: LL – tabuľka použitá pri syntaktickej analýze